1622. Fancy Sequence

Write an API that generates fancy sequences using the append, addAll, and multAll operations.

Implement the Fancy class:

- Fancy() Initializes the object with an empty sequence.
- void append(val) Appends an integer val to the end of the sequence.
- void addAll(inc) Increments all existing values in the sequence by an integer inc.
- void multAll(m) Multiplies all existing values in the sequence by an integer m.
- int getIndex(idx) Gets the current value at index idx (0-indexed) of the sequence **modulo** $10^9 + 7$. If the index is greater or equal than the length of the sequence, return -1.

Example 1:

Input

["Fancy", "append", "addAll", "append", "multAll", "getIndex", "addAll", "append", "multAll", "getIndex", "getIndex", "getIndex"]

[[], [2], [3], [7], [2], [0], [3], [10], [2], [0], [1], [2]]

Output

[null, null, null, null, null, 10, null, null, null, 26, 34, 20]

Explanation

Fancy fancy = new Fancy();

fancy.append(2); // fancy sequence: [2]

fancy.addAll(3); // fancy sequence: [2+3] -> [5]

fancy.append(7); // fancy sequence: [5, 7]

fancy.multAll(2); // fancy sequence: [5*2, 7*2] -> [10, 14]

fancy.getIndex(0); // return 10

fancy.addAll(3); // fancy sequence: [10+3, 14+3] -> [13, 17]

fancy.append(10); // fancy sequence: [13, 17, 10]

fancy.multAll(2); // fancy sequence: [13*2, 17*2, 10*2] -> [26, 34, 20]

fancy.getIndex(0); // return 26

fancy.getIndex(1); // return 34

fancy.getIndex(2); // return 20

Constraints:

- 1 <= val, inc, m <= 100
- 0 <= idx <= 10^5
- At most 10^5 calls total will be made to append, addAll, multAll, and getIndex.

Solution:-

```
import java.util.*;

class Fancy {

    static final int MOD = 1000000007;
    List<Long> arr;
    long mul = 1;
    long add = 0;

    public Fancy() {
        arr = new ArrayList<>();
    }

    public void append(int val) {
        long v = (val - add + MOD) % MOD;
        v = (v * modInverse(mul)) % MOD;
        arr.add(v);
```

```
    }

    public void addAll(int inc) {
        add = (add + inc) % MOD;
    }

    public void multAll(int m) {
        mul = (mul * m) % MOD;
        add = (add * m) % MOD;
    }

    public int getIndex(int idx) {
        if (idx >= arr.size()) return -1;
        return (int)((arr.get(idx) * mul % MOD + add) % MOD);
    }

    private long modInverse(long x) {
        return power(x, MOD - 2);
    }

    private long power(long x, long y) {
        long res = 1;
        x %= MOD;
        while (y > 0) {
            if ((y & 1) == 1) res = (res * x) % MOD;
            x = (x * x) % MOD;
            y >>= 1;
        }
        return res;
    }
}
```

https://leetcode.com/problems/fancy-sequence/?envType=daily-question&envId=2026-03-15

Daily Question | Submit | 89 | Premium

Description | Editorial | Accepted | Solutions | Submission

1622. Fancy Sequence

Solved

Hard | Topics | Companies | Hint

Write an API that generates fancy sequences using the `append`, `addAll`, and `multAll` operations.

Implement the `Fancy` class:

- `Fancy()` Initializes the object with an empty sequence.
- `void append(val)` Appends an integer `val` to the end of the sequence.
- `void addAll(inc)` Increments all existing values in the sequence by an integer `inc`.
- `void multAll(m)` Multiplies all existing values in the sequence by an integer `m`.
- `int getIndex(idx)` Gets the current value at index `idx` (0-indexed) of the sequence **modulo** $10^9 + 7$. If the index is greater or equal than the length of the sequence, return `-1`.

Example 1:

Input
```
["Fancy", "append", "addAll", "append", "multAll",
"getIndex", "addAll", "append", "multAll", "getIndex",
"getIndex", "getIndex"]
[[], [2], [3], [7], [2], [0], [3], [10], [2], [0], [1], [2]]
```
Output
```
[null, null, null, null, null, 10, null, null, null, 26, 34,
20]
```

Explanation

579 | 113 | 4318 Online

Code

Java | Auto

```java
class Fancy {

    static final int MOD = 1000000007;
    List<Long> arr;
    long mul = 1;
    long add = 0;

    public Fancy() {
        arr = new ArrayList<>();
    }

    public void append(int val) {
        long v = (val - add + MOD) % MOD;
        v = (v * modInverse(mul)) % MOD;
        arr.add(v);
    }

    public void addAll(int inc) {
        add = (add + inc) % MOD;
    }

    public void multAll(int m) {
        mul = (mul * m) % MOD;
        add = (add * m) % MOD;
    }

    public int getIndex(int idx) {
        if (idx >= arr.size()) return -1;
        return (int)((arr.get(idx) * mul % MOD + add) % MOD);
    }

    private long modInverse(long x) {
        return power(x, MOD - 2);
    }
```

Saved | Ln 9, Col 1

Testcase | Test Result